

CCP6224 Object Oriented Analysis and Design

Tutorial Zero: Introduction to Java

Lab outcomes

By the end of today's lab, you should be able to

- identify and distinguish between Java command line and graphical applications and Java applets.
- edit, compile and execute Java program code for applications.
- understand basic Java code components and programming requirements

What are Java programs?

Java programs can be broadly categorized into two types, namely (i) applications and (ii) applets. Applications can either run as a command-line program (i.e. without any graphical interface) or with a GUI. Applets run as 'mini' applications within web browsers and are used to enhance the web experience for visitors to a website.

All Java program code have file names that end with **.java** extension. Being an object-oriented language, Java organizes code in a set of classes. As a convention plus a necessity of the compiler/runtime, the name of the file containing a program must exactly match with the class name.

Your first Java (command line) application

Download BlueJ from <https://bluej.org/> and install it on your PC.

Select *New Project* from the menu, and click on the *New Class* button and enter the following program code. The code creates a simple example of a Java application. A Java application is a standalone program that can be executed by using just using the Java runtime interpreter.

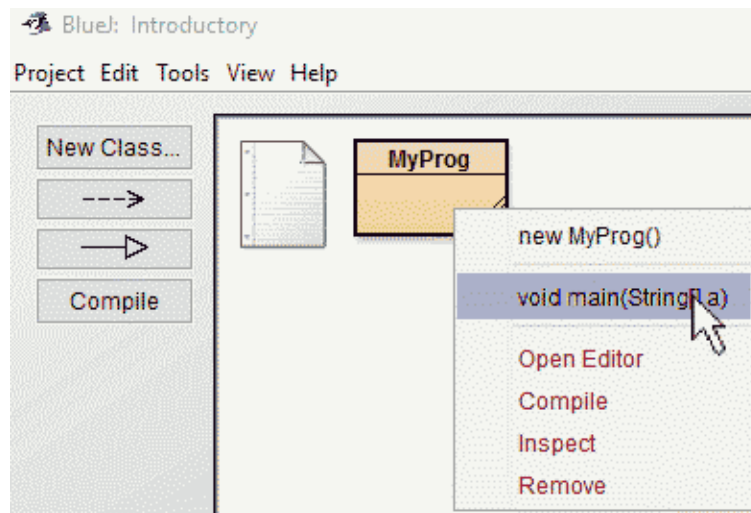
```
public class MyProg
{
    public static void main (String[] a) {
        System.out.println("Hello World!") ;
    }
}
```

Since the above class has a name "**MyProg**", BlueJ will automatically store it in a file called **MyProg.java**. Java is case-sensitive, so capital and small letters are treated differently in the code as well as filenames. If you were not using BlueJ, you would have to manually ensure that this is the case.

The convention in Java is that class names start with capital letters, while variable names start with lower case letters. (For a full list of Java naming conventions, see <http://www.oracle.com/technetwork/java/codeconventions-135099.html>)

Java cannot run program code directly efficiently so it must be compiled – this creates a binary for the platform your program is running on. The same code will compile on Windows, Mac, Linux and other platforms.

To compile in BlueJ, click the *Compile* button, then right-click on the **MyProg** class and select the menu item that contains *main* and click the *Ok* button.



The code could also be compiled and run in the command line as follows:

```
javac myProg.java
```

(Invoking the compiler)

```
java myProg
```

(Invoking run-time interpreter)

```
Hello World!
```

(The output of the program)

If any errors occurred, the compiler or interpreter will exit with an error code, and BlueJ will show you where the error is in the BlueJ editor.

All Java applications require that a **main** function be defined with the attributes **public** and **static**. In the example, **main** is declared as **void** since the method does not return a value. The **main** function expects one (1) argument, a reference to an array of **String** objects – these are arguments passed during runtime. The array holds any command-line arguments passed to the program when it is executed from the command line. To illustrate this feature, modify the program as follows:

```
class ArgProg{
    public static void main(String[] a){
        if(a.length==0)
            System.out.println("Welcome to Java");
        else
            System.out.println("This is the first Java program of "
                               + a[0]);
    }
}
```

1. What is the name of the save file for this program?
2. What is the file output of the compilation process?
3. What does the code statement (**a.length==0**) tell the program?
4. What happens if no arguments are passed?
5. What happens when two or more arguments are passed?

In BlueJ, when you tell it to run, type **{"Adam"}** into the argument box.

For the command line version, compile and run the above application using the command:

java ArgProg

Run the program again with a name as a command line argument.

Example: **java ArgProg Adam**

Compare the output from the programs above.

This feature is mainly useful for using Java programs from the command line. BlueJ is more useful for programs that use the Graphical User Interface (GUI), which is most modern programs.

EXERCISE

- Change the program code so that it displays the number of arguments passed in during program execution.
- Change the program to accept any number of arguments in the command line such as **java ArgProg Martin Kalin** and print out all the passed arguments
In BlueJ, when you tell it to run, type **{"Martin","Kalin"}** into the argument box.

Your first Java GUI application

Java graphical applications are standalone programs that have a user interface (windows, icons, menus and pointers) for user interaction. They do not need to run in a browser but still require the Java runtime to execute. Java graphical applications are similar to command-line applications and **must have a main function** in the code to run. Copy and save the program code below then compile and run the program binary.

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class GUI extends JFrame{
    public GUI(String title){
        super(title);
        setSize(300,100);
        Container content = getContentPane();
        content.setBackground(Color.white);
        content.setLayout(new FlowLayout());
        content.add(new JLabel("Hello world!"));
        setDefaultCloseOperation(EXIT_ON_CLOSE);
    }
    public static void main (String args[]){
        JFrame nframe = new GUI("This is a GUI application");
        nframe.setVisible(true);
    }
}
```

1. What is the command to compile and run the code above?
2. What libraries are imported to create the application?
3. What happens if you add another label "This is my first GUI application" after "Hello World!"? Try resizing the window and see what happens.

EXERCISE

Add this code to your program:

```
public void paint(Graphics g){
    Font f=new Font("Times New Roman",Font.BOLD,20);
    g.setColor(Color.cyan);
    g.fillRect(0,0,800,600);
    g.setFont(f);
    g.setColor(Color.yellow);
    g.fillRect(10,10,400,60);
    g.setColor(Color.pink);
    g.fillRect(10,90,400,60);
    g.setColor(Color.blue);
    g.drawString("Welcome to Java Graphics",20,40);
    g.drawString("Today is "+ new Date(),20,120);
}
```

When you try to compile it, you will get an error because of the use of the Date class from the Java Class Library. Search on <https://docs.oracle.com/en/java/javase/21/docs/api/index.html> for what to import to fix this error.

When you run the program, the window will be too small to show everything and the yellow box will be obscured by the title bar. Adjust the code in the program so that it will show properly, like this:

